Basándonos en el documento que enviaste, tu aplicación móvil (el frontend) definitivamente debe tener más funcionalidades que solo mostrar una predicción de riesgo. No es solo un "panel de control" vacío, sino una herramienta de enseñanza inteligente, interactiva y adaptativa.

Aquí tienes el desglose exacto de lo que debe ir en tu frontend (en Flutter), según el diseño que exige el documento:

1. Integración con Moodle (La capa del LMS)

Cuando el texto dice que la app debe integrarse con Moodle mediante servicios web REST, significa que **sí harás tu propio frontend desde cero**, pero este frontend debe tener pantallas que se comuniquen con Moodle para que el alumno gestione su curso sin tener que abrir el navegador.

Según el documento, tu app necesita pantallas en Flutter para que el estudiante pueda:

- Iniciar sesión usando sus credenciales institucionales.
- Revisar los materiales de apoyo y contenidos de aprendizaje.
- Participar en foros académicos.
- Entregar trabajos y tareas.
- Responder evaluaciones, cuestionarios automáticos y microevaluaciones.

2. La capa de Inteligencia Artificial

El componente de IA es lo que hace que tu sistema sea adaptativo y personalizado. Tu frontend debe mostrar visualmente los resultados de los tres modelos de IA que definimos antes:

- **(Todavía en duda)Sistema de Tutoría Inteligente (El Bot):** La app debe incluir una pantalla de chat conversacional. Este chat debe dar explicaciones en tiempo real y diagnosticar errores, como por ejemplo en patrones de diseño de software (Singleton, Observer, Factory).
- **(Todavía en duda) Trayectorias Personalizadas (Recomendaciones):** La app debe tener una sección que muestre rutas de aprendizaje y módulos sugeridos adaptados a las necesidades específicas del estudiante.
- **Riesgo Predictivo y Retroalimentación:** La app debe tener un panel de progreso interactivo. Aquí se muestra la predicción del riesgo de bajo rendimiento , alertas sobre vacíos de comprensión y retroalimentación automática sobre lo que el alumno está haciendo bien o mal.

3. Estrategias Pedagógicas (Más pantallas y elementos visuales)

El documento exige que el frontend incorpore funciones para apoyar cinco estrategias metodológicas:

- **Ejercitación:** Pantallas para resolver ejercicios de programación y cuestionarios que te den los resultados inmediatamente.
- **Elaboración:** Funciones para ver resúmenes guiados, mapas conceptuales o participar en foros.

- **Organización:** Interfaces que muestren esquemas visuales de patrones de diseño y módulos temáticos estructurados.
- **Control Metacognitivo:** Un *dashboard* o panel con estadísticas de la actividad del alumno y notificaciones/alertas de cumplimiento de tareas.
- **Gamificación:** Elementos visuales muy propios de una app móvil, como mostrar puntos acumulados, insignias ganadas, niveles de logro, retos y un ranking de estudiantes.

—----------

Acotar el alcance a una sola funcionalidad de Inteligencia Artificial (el Modelo Predictivo de Riesgo) hace que el proyecto sea mucho más realista para este ciclo de desarrollo.

Al descartar por ahora el Chatbot (tutor virtual) y el Sistema de Recomendación de contenido, el alcance de tu frontend en Flutter se simplifica y se vuelve mucho más directo.

Así es exactamente como debe quedar estructurada tu aplicación móvil para presentar este avance, enfocándote al 100% en la predicción de riesgo y cumpliendo con el modelo de la lectura:

Las Pantallas Clave para el Frontend (Flutter + MVVM)

1. Pantalla de Autenticación (Login)

- **Qué hace:** El estudiante ingresa con sus credenciales universitarias.
- **Integración:** Se conecta a tu backend para validar los datos contra la base de datos (PostgreSQL/MySQL).

2. Pantalla Principal (Dashboard de Cursos)

- **Qué hace:** Muestra la lista de los cursos en los que el alumno está matriculado.
- **Integración:** Aquí consumes un endpoint básico que lee la información de Moodle. No necesitas descargar los PDFs ni los foros todavía, solo mostrar las tarjetas con los nombres de los cursos para cumplir con la dimensión de "Aplicaciones".

3. Panel de Progreso y Riesgo Académico (La pantalla estrella)

- **Qué hace:** Esta es la interfaz gráfica de los algoritmos que programó tu compañero. Al entrar a un curso específico, el alumno no ve un simple listado de tareas, sino un **Dashboard de Riesgo**.
- **Diseño visual:** Puedes usar un gráfico de medidor (tipo velocímetro) o un semáforo (Verde = Seguro, Amarillo = En riesgo, Rojo = Riesgo crítico). Si el algoritmo (ej. Random Forest) devuelve un "85% de probabilidad de reprobar", la pantalla se pone en rojo y emite la alerta temprana.

4. Pantalla de Control Metacognitivo (El "Por qué" del riesgo)

- **Qué hace:** Para que la predicción no sea una "caja negra" que solo asusta al alumno, debes mostrarle estadísticas de su propia actividad.

- **Integración:** Muestras los datos transaccionales que sacaron de Moodle (ej. "Llevas 5 días sin entrar al sistema", "Tu promedio en los últimos cuestionarios es 11"). Esto justifica el porcentaje de riesgo y ayuda al estudiante a autorregularse.

Cómo orquestar esto a nivel de Arquitectura

Para que Flutter no haga el trabajo pesado, el flujo ideal es mantener la lógica separada. en PostgreSQL y enviarle esos datos al entorno de Python donde viven sus modelos de Machine Learning. y fastAPI para el backend

Finalmente, tu backend en Java simplemente expone un endpoint REST (por ejemplo, GET /api/riesgo/alumno/123) que devuelve un JSON limpio. Tu aplicación en Flutter consume ese JSON y actualiza el medidor visual en la vista usando el patrón MVVM.